

HƯỚNG DẪN CHỨC NĂNG LOGIN BẰNG SPRING SECURITY 7 + SPRING BOOT+MAPSTRUCT

Ví dụ 1: Cho bảng User, Role hãy viết chức năng login, thông tin của user sẽ hiển thị ở header.html. Sử dụng spring boot 4 và spring security, mapstruct, thymeleaf, layout không dùng Dialect.

Bước 1: Thêm thư viện vào file pom.xml sau khi tạo project

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>vn.iotstar</groupId>
  <artifactId>springboot1-8</artifactId>
  <version>1.0</version>
  <name>springboot1-8</name>
  <description/>
  <url/>
  <licenses>
    <license/>
  </licenses>
  <developers>
    <developer/>
  </developers>
  <scm>
    <connection/>
    <developerConnection/>
    <tag/>
    <url/>
  </scm>
  <properties>
    <java.version>26</java.version>
    <mapstruct.version>1.6.3</mapstruct.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-h2console</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
  </dependencies>
```

```

        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-mail</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>
    <dependency>
        <groupId>org.thymeleaf.extras</groupId>
        <artifactId>thymeleaf-extras-springsecurity6</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>com.microsoft.sqlserver</groupId>
        <artifactId>mssql-jdbc</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-mail-test</artifactId>
        <scope>test</scope>
    </dependency>

```

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>com.cloudinary</groupId>
  <artifactId>cloudinary-http5</artifactId>
  <version>2.4.0</version>
</dependency>
<dependency>
  <groupId>org.mapstruct</groupId>
  <artifactId>mapstruct</artifactId>
  <version>${mapstruct.version}</version>
</dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <configuration>
        <release>${java.version}</release>

        <annotationProcessorPaths>
          <path>
            <groupId>org.mapstruct</groupId>
            <artifactId>mapstruct-processor</artifactId>
            <version>${mapstruct.version}</version>
          </path>

          <path>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
          </path>

          <path>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok-mapstruct-binding</artifactId>

```

```

        <version>0.2.0</version>
    </path>
</annotationProcessorPaths>
</configuration>
</plugin>
</plugins>
</build>

```

```
</project>
```

Bước 2: Cấu hình file application.properties

```

spring.config.import=optional:file:.env[.properties]
spring.application.name=springboot1-8
server.port=${SERVER_PORT:8088}

spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}

spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.format_sql=true
spring.datasource.driverClassName=com.microsoft.sqlserver.jdbc.SQLServerDriver

spring.thymeleaf.cache=false
spring.thymeleaf.encoding=UTF-8

spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=20MB

spring.mail.host=${MAIL_HOST}
spring.mail.port=${MAIL_PORT}
spring.mail.username=${MAIL_USERNAME}
spring.mail.password=${MAIL_PASSWORD}
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
spring.mail.properties.mail.smtp.connectiontimeout=5000
spring.mail.properties.mail.smtp.timeout=5000
spring.mail.properties.mail.smtp.writetimeout=5000

cloudinary.cloud-name=${CLOUDINARY_CLOUD_NAME}
cloudinary.api-key=${CLOUDINARY_API_KEY}
cloudinary.api-secret=${CLOUDINARY_API_SECRET}

spring.servlet.encoding.enabled=true
spring.servlet.encoding.charset=UTF-8
spring.servlet.encoding.force=true
spring.servlet.encoding.force-request=true
spring.servlet.encoding.force-response=true

spring.main.allow-bean-definition-overriding=true

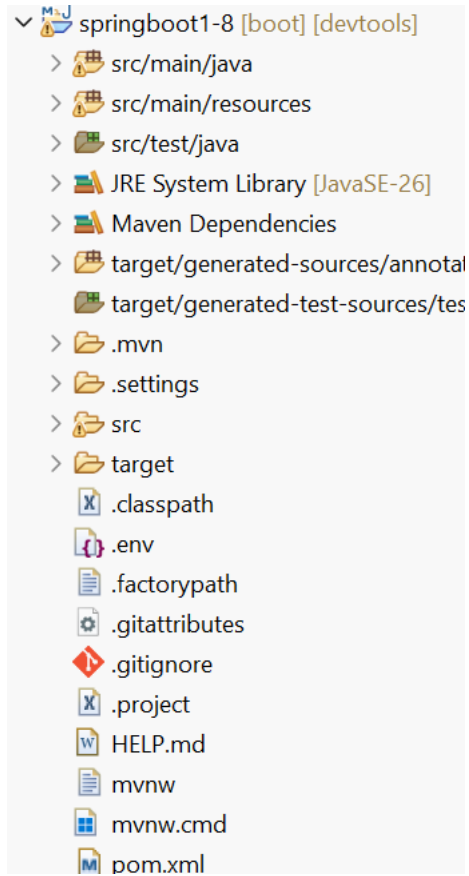
```

Bước 3: Cấu hình biến môi trường .env

- Thêm dòng sau vào đầu của file application.properties nếu chưa có:

spring.config.import=optional:file:.env[.properties]

- Tạo file .env tại project ngang cấp với file pom.xml như hình:



- Soạn thảo file .env như sau và thay đổi thông số phù hợp với máy mình

```
# =====
# DATABASE
# =====
DB_URL=jdbc:sqlserver://localhost:1433;databaseName=webst4;encrypt=false;trustServerCertificate=true;sslProtocol=TLSv1.2;characterEncoding=UTF-8
DB_USERNAME=sa
DB_PASSWORD=13234

# =====
# JPA
# =====
DDL_AUTO=update
SHOW_SQL=true

# =====
# SERVER
# =====
SERVER_PORT=8080
```

```
# =====
# JWT
# =====
#JWT_SECRET=your-super-secret-key-change-this-in-production
#JWT_ACCESS_EXPIRATION=900000
#JWT_REFRESH_EXPIRATION=604800000

# =====
# SMTP
# =====
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=dfdfdf
MAIL_PASSWORD=zkhxpxfcalaknld

# =====
# CLOUDINARY
# =====
CLOUDINARY_CLOUD_NAME=dfdfdf
CLOUDINARY_API_KEY=576632571682623
CLOUDINARY_API_SECRET=ikPEbngxnKwAw-XkvR1WVEaQZcI
```

- Mở file .gitignore (nếu có) và thêm .env vào file .gitignore để khi up lên github sẽ không up file .env

Bước 4: Tạo class mã hóa UTF-8 trong spring boot

```
package vn.iotstar.config;

import org.springframework.boot.web.servlet.FilterRegistrationBean;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.filter.CharacterEncodingFilter;

@Configuration
public class EncodingConfig {
    @Bean
    FilterRegistrationBean<CharacterEncodingFilter> characterEncodingFilter() {
        CharacterEncodingFilter filter = new CharacterEncodingFilter();
        filter.setEncoding("UTF-8");
        filter.setForceEncoding(true);
        FilterRegistrationBean<CharacterEncodingFilter> registration = new
        FilterRegistrationBean<>(filter);
        registration.setOrder(Integer.MIN_VALUE);
        return registration;
    }
}
```

Bước 5: Tạo class cấu hình Spring security

```
package vn.iotstar.config;

import org.springframework.context.annotation.*;
```

```

import
org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
@Configuration
@EnableMethodSecurity

public class SecurityConfig {

    @Bean
    PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/login", "/register", "/verify-otp", "/forgot-
password",
                    "/reset-password", "/register/resend-otp", "/css/**",
"/images/**", "/error").permitAll()
                .requestMatchers("/users/**", "/dashboard").hasRole("ADMIN")
                .requestMatchers("/categories/**").authenticated()
                .requestMatchers("/products/**").authenticated()
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/login")
                .loginProcessingUrl("/login")
                .usernameParameter("email")
                .passwordParameter("password")
                .defaultSuccessUrl("/dashboard", true)
                .failureUrl("/login?error=true")
                .permitAll()
            )
            .logout(logout -> logout
                .logoutUrl("/logout")
                .logoutSuccessUrl("/login?logout=true")
                .invalidateHttpSession(true)
                .deleteCookies("JSESSIONID")
                .permitAll()
            )
            .exceptionHandling(ex -> ex.accessDeniedPage("/access-denied"));
        return http.build();
    }
}

```

Bước 6: Tạo class cấu hình Login với email

```
package vn.iotstar.security;
```

```

import org.springframework.security.core.userdetails.*;
import org.springframework.stereotype.Service;
import vn.iotstar.entity.User;
import vn.iotstar.repository.UserRepository;

@Service
public class CustomUserDetailsService implements UserDetailsService {
    private final UserRepository users;
    public CustomUserDetailsService(UserRepository users) { this.users=users; }

    @Override
    public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
        User u=users.findByEmailWithRole(username)
            .orElseThrow(() -> new UsernameNotFoundException("Không tìm thấy tài
khoản."));
        return
org.springframework.security.core.userdetails.User.withUsername(u.getEmail())
            .password(u.getPassword())
            .roles(u.getRole().getName())
            .disabled(!u.isEnabled())
            .build();
    }
}

```

Bước 7: Tạo Entity

```

package vn.iotstar.entity;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.util.ArrayList;
import java.util.List;

@Data
@AllArgsConstructor
@NoArgsConstructor
@Entity
@Table(name = "roles")
public class Role {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 30)
    private String name;

    @OneToMany(mappedBy = "role")
    private List<User> users = new ArrayList<>();

    public Role(String name) { this.name = name; }
}

```

```

package vn.iotstar.entity;

```

```

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

@Data
@AllArgsConstructor
@NoArgsConstructor
@Entity
@Table(name = "users")
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 120)
    private String email;

    @Column(nullable = false, length = 150)
    private String password;

    @Column(nullable = false, length = 120, columnDefinition = "nvarchar(120)")
    private String fullName;

    @Column(nullable = false)
    private boolean enabled = false;

    @Column(nullable = false)
    private LocalDateTime createdAt = LocalDateTime.now();

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "role_id", nullable = false)
    private Role role;

    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Product> products = new ArrayList<>();
}

```

Bước 8: Tạo DTOs

```

package vn.iotstar.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import lombok.Data;

import java.time.LocalDateTime;

@Data

```

```

public class UserDTO {
    private Long id;
    @Email @NotBlank
    private String email;
    @NotBlank
    private String fullName;
    @NotNull
    private Long roleId;
    private String roleName;
    private boolean enabled;
    private long productCount;
    private LocalDateTime createdAt;
}
}

package vn.iotstar.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import lombok.Data;

@Data
public class LoginDTO {
    @Email @NotBlank
    private String email;
    @NotBlank
    @Size(min=6, max=100)
    private String password;
}

```

Bước 9: Tạo interface mapper bằng mapstruct

```

package vn.iotstar.mapper;

import org.mapstruct.*;
import vn.iotstar.dto.UserDTO;
import vn.iotstar.entity.User;

@Mapper(componentModel = "spring", unmappedTargetPolicy = ReportingPolicy.IGNORE)
public interface UserMapper {
    @Mapping(target="roleId", source="role.id")
    @Mapping(target="roleName", source="role.name")
    UserDTO toDto(User entity);

    @Mapping(target="role", ignore=true)
    @Mapping(target="products", ignore=true)
    User toEntity(UserDTO dto);
}

```

Bước 10: Tạo interface repository

```

package vn.iotstar.repository;

import vn.iotstar.entity.Role;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

```

```

public interface RoleRepository extends JpaRepository<Role, Long> {

    Optional<Role> findByNameIgnoreCase(String name);

}
}

package vn.iotstar.repository;

import vn.iotstar.entity.User;
import org.springframework.data.domain.*;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByEmailIgnoreCase(String email);

    boolean existsByEmailIgnoreCase(String email);

    Page<User> findByEmailContainingIgnoreCaseOrFullNameContainingIgnoreCase(
        String email, String fullName, Pageable pageable);

    @Query("""
        SELECT u
        FROM User u
        JOIN FETCH u.role
        WHERE u.email = :email
        """)
    Optional<User> findByEmailWithRole(
        @Param("email") String email
    );

}

```

Bước 11: Tạo Controller

```

package vn.iotstar.controller;
import jakarta.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import vn.iotstar.dto.*;
import vn.iotstar.service.AuthService;

@Controller
public class AuthController {
    private final AuthService auth;
    public AuthController(AuthService auth) { this.auth=auth; }

    @GetMapping("/login")
    String login() { return "auth/login"; }
}

```

Phương thức post của Login sẽ được Spring Security tự sinh ra.

Bước 12: Tạo dữ liệu mẫu

```
package vn.iotstar.config;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.*;
import org.springframework.security.crypto.password.PasswordEncoder;
import vn.iotstar.entity.*;
import vn.iotstar.entity.Role;
import vn.iotstar.repository.*;

@Configuration
public class DataInitializer {
    @Bean
    CommandLineRunner initData(RoleRepository roles, UserRepository users,
                               CategoryRepository categories, PasswordEncoder
encoder,
                               @Value("${ADMIN_EMAIL: trungnh@hcmute.edu.vn}") String
adminEmail,
                               @Value("${ADMIN_PASSWORD: 123456}") String
adminPassword) {
        return args -> {
            Role userRole=roles.findByNameIgnoreCase("USER").orElseGet(() ->
roles.save(new Role("USER")));
            Role adminRole=roles.findByNameIgnoreCase("ADMIN").orElseGet(() ->
roles.save(new Role("ADMIN")));

            if (!users.existsByEmailIgnoreCase(adminEmail)) {
                User admin=new User();
                admin.setEmail(adminEmail.toLowerCase());
                admin.setFullName("System Administrator");
                admin.setPassword(encoder.encode(adminPassword));
                admin.setRole(adminRole);
                admin.setEnabled(true);
                users.save(admin);
            }

        }
    }
}
```

Bước 13: Tạo view login

```
<!DOCTYPE html>
<html lang="vi" xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{layouts/layout :: head('Đăng nhập')}"></head>
<body class="auth-page">
    <div class="auth-card">
        <h1>IOTSTAR SHOP</h1>
        <h2>Đăng nhập</h2>
        <div class="alert error" th:if="${param.error}>Email hoặc mật
            khẩu không đúng.</div>
        <div class="alert success" th:if="${param.verified}>Xác thực
```

```

        thành công. Hãy đăng nhập.</div>
<div class="alert success" th:if="${param.reset}">Đổi mật khẩu
    thành công.</div>
<div class="alert success" th:if="${param.logout}">Bạn đã đăng
    xuất.</div>
<form th:action="@{/login}" method="post">
    <label>Email</label><input type="email" name="email" required
        autofocus> <label>Mật khẩu</label><input type="password"
            name="password" required> <input type="hidden"
                th:name="${_csrf.parameterName}"
th:value="${_csrf.token}">
    <button class="btn primary full" type="submit">Đăng nhập</button>
</form>
<p>
    <a th:href="@{/register}">Tạo tài khoản</a> . <a
        th:href="@{/forgot-password}">Quên mật khẩu?</a>
    </p>
</div>
</body>
</html>

```

Layouts/layout.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:fragment="head(title)">
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title th:text="${title}">IOTSTAR SHOP</title>
<link rel="stylesheet" th:href="@{/css/app.css}">
</head>
</html>

```

Fragments/header.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
    xmlns:sec="http://www.thymeleaf.org/extras/spring-security">

<body>

<header th:fragment="header" class="topbar">

    <div class="brand">
        <a th:href="@{/}">
            IOTSTAR SHOP
        </a>
    </div>

    <div class="account" sec:authorize="isAuthenticated()">

        <span sec:authentication="name">
            user
        </span>

        <form th:action="@{/Logout}"

```

```

        method="post"
        style="display:inline">

        <input type="hidden"
            th:name="${_csrf.parameterName}"
            th:value="${_csrf.token}">

        <button class="link-button"
            type="submit">
            Đăng xuất
        </button>
    </form>
</div>
</header>
</body>
</html>

```

=====

Home.html

```

<!DOCTYPE html>
<html lang="vi"
    xmlns:th="http://www.thymeleaf.org">
<body>
<header th:replace="~{fragments/header :: header}"></header>
<main class="container">
    <h1>Home</h1>
</main>

</body>
</html>

```

=====

Static/css/app.css

```

* {
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: Arial, Helvetica, sans-serif;
    background: #f5f7fb;
    color: #172033;
}

.topbar {
    height: 64px;
    background: #172554;
    color: #fff;
    display: flex;
    align-items: center;
    padding: 0 28px;
    gap: 30px
}

```

```

}

.brand a {
  color: #fff;
  text-decoration: none;
  font-weight: 800;
  font-size: 20px
}

.topbar nav {
  display: flex;
  gap: 18px;
  flex: 1
}

.topbar nav a {
  color: #dbeafe;
  text-decoration: none
}

.account {
  white-space: nowrap
}

.Link-button {
  background: none;
  border: 0;
  color: #fff;
  cursor: pointer
}

.container {
  max-width: 1200px;
  margin: 30px auto;
  padding: 0 20px
}

.narrow {
  max-width: 700px
}

.page-title {
  display: flex;
  align-items: center;
  justify-content: space-between
}

.panel, .card {
  background: #fff;
  border-radius: 14px;
  padding: 22px;
  box-shadow: 0 4px 18px rgba(0, 0, 0, .06)
}

.cards {

```

```
        display: grid;
        grid-template-columns: repeat(4, 1fr);
        gap: 18px
    }

    .card b {
        display: block;
        color: #64748b
    }

    .card strong {
        display: block;
        font-size: 34px;
        margin-top: 10px
    }

    .btn {
        display: inline-block;
        border: 0;
        border-radius: 8px;
        padding: 9px 14px;
        text-decoration: none;
        cursor: pointer;
        font-weight: 600
    }

    .primary {
        background: #2563eb;
        color: #fff
    }

    .secondary {
        background: #e2e8f0;
        color: #172033
    }

    .danger {
        background: #dc2626;
        color: #fff
    }

    .small {
        padding: 6px 9px;
        font-size: 13px
    }

    .full {
        width: 100%
    }

    .search {
        display: flex;
        gap: 10px;
        margin: 18px 0
    }
```

```
.search input {
    flex: 1
}

.search input, .form-grid input, .form-grid select, .form-grid textarea,
.auth-card input {
    width: 100%;
    padding: 10px;
    border: 1px solid #cbd5e1;
    border-radius: 8px
}

.table-wrap {
    overflow: auto;
    background: #fff;
    border-radius: 12px
}

.table-wrap table {
    width: 100%;
    border-collapse: collapse
}

.table-wrap th, .table-wrap td {
    padding: 12px;
    border-bottom: 1px solid #e2e8f0;
    text-align: left;
    vertical-align: middle
}

.thumb {
    width: 64px;
    height: 64px;
    object-fit: cover;
    border-radius: 8px
}

.form-grid {
    display: grid;
    grid-template-columns: 150px 1fr;
    gap: 14px;
    align-items: center
}

.form-grid textarea {
    resize: vertical
}

.form-grid .hint, .form-grid div {
    grid-column: 2
}

.alert {
    padding: 12px;
```

```
        border-radius: 8px;
        margin: 12px 0
    }

    .error {
        background: #fee2e2;
        color: #991b1b
    }

    .success {
        background: #dcfce7;
        color: #166534
    }

    .field-error {
        color: #b91c1c
    }

    .pagination {
        display: flex;
        gap: 6px;
        justify-content: center;
        margin: 22px
    }

    .pagination a {
        display: inline-block;
        padding: 7px 11px;
        background: #fff;
        border-radius: 7px;
        text-decoration: none;
        color: #172033
    }

    .pagination .active a {
        background: #2563eb;
        color: #fff
    }

    .footer {
        text-align: center;
        padding: 30px;
        color: #64748b
    }

    .auth-page {
        min-height: 100vh;
        display: grid;
        place-items: center;
        background: linear-gradient(135deg, #eff6ff, #eef2ff)
    }

    .auth-card {
        width: min(440px, 92vw);
        background: #fff;
```

```

        padding: 30px;
        border-radius: 18px;
        box-shadow: 0 15px 50px rgba(15, 23, 42, .12)
    }

    .auth-card h1 {
        color: #1d4ed8
    }

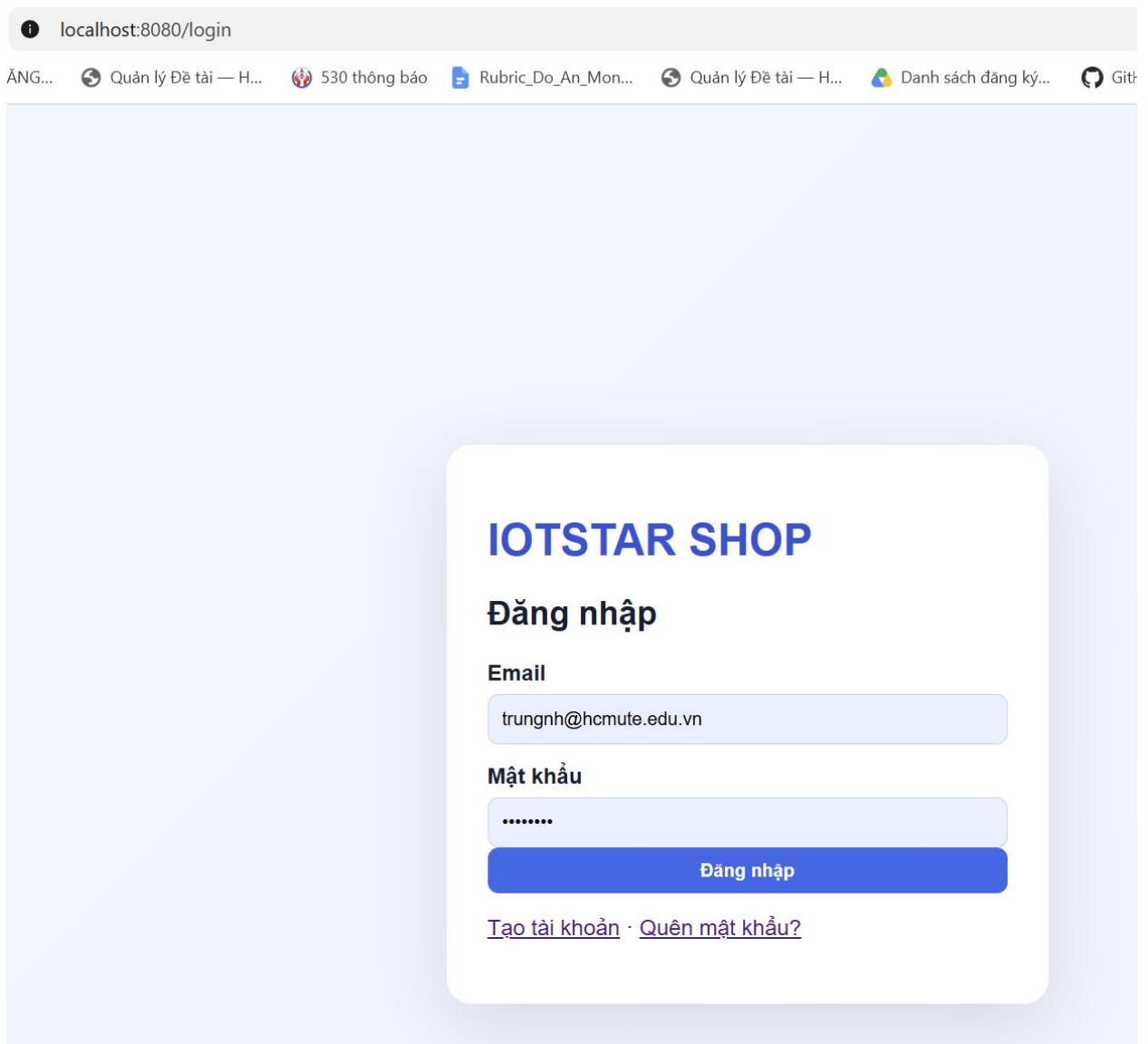
    .auth-card label {
        display: block;
        margin: 14px 0 6px;
        font-weight: 600
    }

    .mt {
        margin-top: 10px
    }

    .actions {
        white-space: nowrap
    }

    @media ( max-width :800px) {
        .cards {
            grid-template-columns: 1fr 1fr
        }
        .topbar {
            padding: 0 12px;
            gap: 12px
        }
        .topbar nav {
            gap: 8px;
            font-size: 13px
        }
        .form-grid {
            grid-template-columns: 1fr
        }
        .form-grid .hint, .form-grid div {
            grid-column: 1
        }
    }

```



Ví dụ 2: Cho bảng User, Role hãy viết chức năng Custom login có thể đăng nhập bằng username hoặc email đều được, thông tin fullname và images của user sẽ hiển thị ở header.html. Sử dụng spring boot 4 và spring security, mapstruct, thymeleaf với Dialect.

Bước 1: Thêm thư viện vào file pom.xml sau khi tạo project có security và thymeleaf.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
```

```

        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>vn.iotstar</groupId>
    <artifactId>springboot1-9</artifactId>
    <version>1.0</version>
    <name>springboot1-9</name>
    <description/>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>26</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-h2console</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-mail</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-thymeleaf</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>
        <dependency>
            <groupId>org.thymeleaf.extras</groupId>
            <artifactId>thymeleaf-extras-springsecurity6</artifactId>

```

```

</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>com.microsoft.sqlserver</groupId>
  <artifactId>mssql-jdbc</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-mail-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.mapstruct</groupId>

```

```

        <artifactId>mapstruct</artifactId>
        <version>1.6.3</version>
    </dependency>
    <dependency>
        <groupId>nz.net.ultraq.thymeleaf</groupId>
        <artifactId>thymeleaf-layout-dialect</artifactId>
        <scope>compile</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <executions>
                <execution>
                    <id>default-compile</id>
                    <phase>compile</phase>
                    <goals>
                        <goal>compile</goal>
                    </goals>
                    <configuration>
                        <annotationProcessorPaths>
                            <path>
                                <groupId>org.mapstruct</groupId>
                                <artifactId>mapstruct-processor</artifactId>
                                <version>1.6.3</version>
                            </path>
                            <path>
                                <groupId>org.projectlombok</groupId>
                                <artifactId>lombok-mapstruct-binding</artifactId>
                                <version>0.2.0</version>
                            </path>
                        </annotationProcessorPaths>
                    </configuration>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>

<groupId>org.projectlombok</groupId>
<artifactId>lombok</artifactId>
</path>
<path>

<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-
configuration-processor</artifactId>
</path>
</annotationProcessorPaths>
</configuration>
</execution>
<execution>
    <id>default-testCompile</id>

```

```

        <phase>test-compile</phase>
        <goals>
            <goal>testCompile</goal>
        </goals>
        <configuration>
            <annotationProcessorPaths>
                <path>

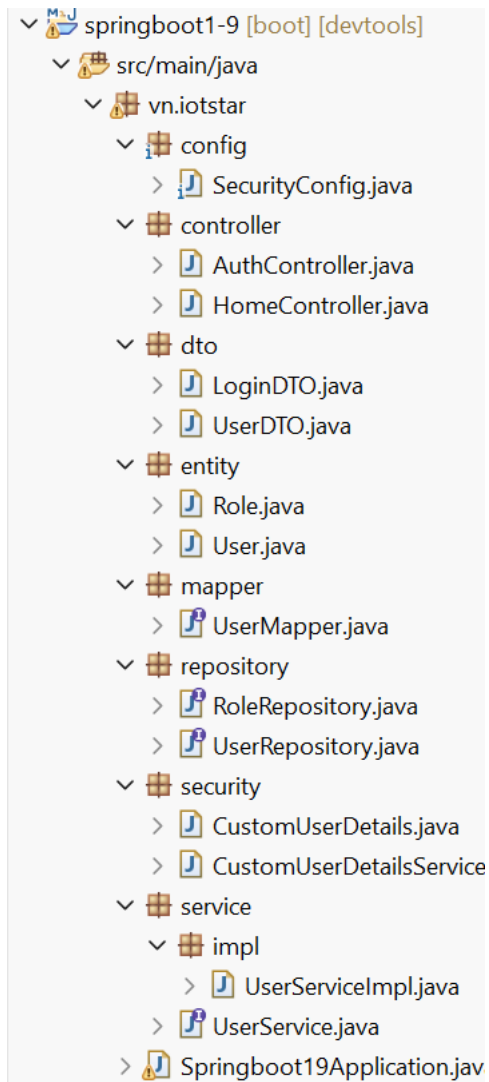
<groupId>org.projectlombok</groupId>

<artifactId>lombok</artifactId>

                </path>
            </annotationProcessorPaths>
        </configuration>
    </execution>
</executions>
</plugin>
</plugins>
</build>

</project>

```



Bước 2: cấu hình file Application.properties không dùng file môi trường .env. Nếu muốn cấu hình môi trường .env thì làm tương tự bước 2 ở ví dụ 1.

```
spring.application.name=springboot1-9
server.port=8081
```

```
# Thymeleaf
#spring.thymeleaf.prefix=classpath:/templates/
#spring.thymeleaf.suffix=.html
spring.thymeleaf.cache=false
```

```
#kết nối database
spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=webst9;encrypt=false;trustServerCertificate=true;sslProtocol=TLSv1.2;characterEncoding=UTF-8
spring.datasource.username=sa
spring.datasource.password=1234567@a$
spring.datasource.driverClassName=com.microsoft.sqlserver.jdbc.SQLServerDriver
```

```
#JPA
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.format_sql=true
#Hibernate ddl auto (create, create-drop, update, none)
spring.jpa.hibernate.ddl-auto=update

spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.SQLServerDialect
```

```
# =====
# FILE UPLOAD
# =====
```

```
spring.servlet.multipart.enabled=true

spring.servlet.multipart.max-file-size=5MB

spring.servlet.multipart.max-request-size=10MB
```

Bước 3: Cấu hình Security cho phép đăng nhập từ email hoặc username đều được

```
package vn.iotstar.config;

import lombok.RequiredArgsConstructor;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;

import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;

import org.springframework.security.config.annotation.web.builders.HttpSecurity;

import org.springframework.security.core.userdetails.UserDetailsService;

import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final UserDetailsService userDetailsService;

    @Bean
    public PasswordEncoder passwordEncoder() {
```

```

        return new BCryptPasswordEncoder();
    }

    @Bean
    public DaoAuthenticationProvider authenticationProvider() {

        DaoAuthenticationProvider provider =
            new DaoAuthenticationProvider(userDetailsService);

        provider.setPasswordEncoder(passwordEncoder());

        return provider;
    }

    @Bean
    public AuthenticationManager authenticationManager(
        AuthenticationConfiguration configuration
    ) throws Exception {

        return configuration.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(
        HttpSecurity http
    ) throws Exception {

        http
            .authorizeHttpRequests(auth -> auth

                .requestMatchers(
                    "/login",
                    "/css/**",
                    "/js/**",
                    "/images/**",
                    "/uploads/**"
                ).permitAll()

                .requestMatchers("/admin/**")
                .hasRole("ADMIN")

                .anyRequest()
                .authenticated()
            )

            .formLogin(form -> form

                .loginPage("/login")

                .loginProcessingUrl("/login")

                .defaultSuccessUrl(
                    "/",
                    true
                )
            )
    }

```

```

        .failureUrl(
            "/login?error=true"
        )
        .permitAll()
    )
    .logout(logout -> logout
        .logoutUrl("/logout")
        .logoutSuccessUrl("/login?logout=true")
        .invalidateHttpSession(true)
        .deleteCookies("JSESSIONID")
        .permitAll()
    );

    return http.build();
}
}

```

```

package vn.iotstar.security;

import lombok.RequiredArgsConstructor;

import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import vn.iotstar.entity.User;
import vn.iotstar.repository.UserRepository;

@Service
@RequiredArgsConstructor
public class CustomUserDetailsService
    implements UserDetailsService {

    private final UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String login)
        throws UsernameNotFoundException {

        User user = userRepository
            .findByUsernameOrEmail(login, login)
            .orElseThrow(() ->
                new UsernameNotFoundException(
                    "Không tìm thấy username/email: " + login
                )
            );
    }
}

```

```

        return new CustomUserDetails(
            user.getId(),
            user.getUsername(),
            user.getEmail(),
            user.getPassword(),
            user.getFullName(),
            user.getImages(),
            user.getRole().getName(),
            user.isEnabled()
        );
    }
}

```

Tạo CustomUser để tùy chỉnh được thông tin User khi đưa ra views

```

package vn.iotstar.security;

import java.util.Collection;
import java.util.List;

import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;

import lombok.Getter;

@Getter
public class CustomUserDetails implements UserDetails {

    private static final long serialVersionUID = 1L;

    private final Long id;

    private final String username;

    private final String email;

    private final String password;

    private final String fullName;

    private final String images;

    private final String role;

    private final boolean enabled;

    public CustomUserDetails(
        Long id,
        String username,
        String email,
        String password,
        String fullName,

```

```

        String images,
        String role,
        boolean enabled
    ) {
        this.id = id;
        this.username = username;
        this.email = email;
        this.password = password;
        this.fullName = fullName;
        this.images = images;
        this.role = role;
        this.enabled = enabled;
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {

        return List.of(
            new SimpleGrantedAuthority(role)
        );
    }

    @Override
    public String getPassword() {
        return password;
    }

    @Override
    public String getUsername() {
        return username;
    }

    @Override
    public boolean isAccountNonExpired() {
        return true;
    }

    @Override
    public boolean isAccountNonLocked() {
        return true;
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return true;
    }

    @Override
    public boolean isEnabled() {
        return enabled;
    }
}

```

Bước 4: tạo Entity

```
package vn.iotstar.entity;
```

```

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "roles")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Role {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String name;
}

```

```
package vn.iotstar.entity;
```

```

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(
    name = "users",
    uniqueConstraints = {
        @UniqueConstraint(name = "uk_users_username",
            columnNames = "username"),
        @UniqueConstraint(name = "uk_users_email",
            columnNames = "email")
    }
)
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String username;

    @Column(nullable = false, unique = true, length = 150)
    private String email;

    @Column(nullable = false)
    private String password;
}

```

```

@Column(name = "full_name", length = 150, columnDefinition = "nvarchar(200)")
private String fullName;

@Column(length = 500)
private String images;

@Column(nullable = false)
@Builder.Default
private boolean enabled = true;

@ManyToOne(fetch = FetchType.EAGER)
@JoinColumn(name = "role_id", nullable = false)
private Role role;
}

```

Bước 5: Tạo DTOs

```
package vn.iotstar.dto;
```

```
import lombok.*;
```

```

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class UserDTO {

    private Long id;

    private String username;

    private String email;

    private String fullName;

    private String images;

    private String roleName;

    private boolean enabled;
}

```

```
package vn.iotstar.dto;
```

```
import jakarta.validation.constraints.NotBlank;
import lombok.*;
```

```

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class LoginDTO {

    @NotBlank(message = "Username hoặc email không được để trống")
    private String login;
}

```

```

        @NotBlank(message = "Password không được để trống")
        private String password;
    }

```

Bước 6: Tạo mapper với mapstruct

```

package vn.iotstar.mapper;

import org.mapstruct.Mapper;
import org.mapstruct.Mapping;

import vn.iotstar.dto.UserDTO;
import vn.iotstar.entity.User;

@Mapper(componentModel = "spring")
public interface UserMapper {

    @Mapping(
        target = "roleName",
        source = "role.name"
    )
    UserDTO toDTO(User user);
}

```

Bước 7: Tạo repository

```

package vn.iotstar.repository;

import vn.iotstar.entity.Role;

import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;

public interface RoleRepository extends JpaRepository<Role, Long> {

    Optional<Role> findByName(String name);
}

```

```

package vn.iotstar.repository;

import vn.iotstar.entity.User;

import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByUsername(String username);

    Optional<User> findByEmail(String email);
}

```

```

        Optional<User> findByUsernameOrEmail(
            String username,
            String email
        );
    }

```

Bước 8: tạo Service

```

package vn.iotstar.service;

import vn.iotstar.dto.UserDTO;

public interface UserService {

    UserDTO findById(Long id);

}

=====

package vn.iotstar.service.impl;

import org.springframework.stereotype.Service;

import lombok.RequiredArgsConstructor;
import vn.iotstar.dto.UserDTO;
import vn.iotstar.entity.User;
import vn.iotstar.mapper.UserMapper;
import vn.iotstar.repository.UserRepository;
import vn.iotstar.service.UserService;

@Service
@RequiredArgsConstructor
public class UserServiceImpl implements UserService {

    private final UserRepository userRepository;

    private final UserMapper userMapper;

    @Override
    public UserDTO findById(Long id) {

        User user = userRepository.findById(id)
            .orElseThrow();

        return userMapper.toDTO(user);
    }
}

```

Bước 9: Tạo Controller

```

package vn.iotstar.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller

```

```

public class AuthController {

    @GetMapping("/login")
    public String login() {
        return "auth/login";
    }
}
}

package vn.iotstar.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class HomeController {

    @GetMapping("/")
    public String home() {
        return "home";
    }
}
}

```

Bước 10: Tạo dữ liệu mẫu, tạo Bean trongApplication.java

```

@Bean
CommandLineRunner init(
    RoleRepository roleRepository,
    UserRepository userRepository,
    PasswordEncoder passwordEncoder
) {

    return args -> {

        Role userRole = roleRepository
            .findByName("ROLE_USER")
            .orElseGet(() ->
                roleRepository.save(
                    Role.builder()
                        .name("ROLE_USER")
                        .build()
                )
            );

        if (userRepository
            .findByUsername("user01")
            .isEmpty()) {

            User user = User.builder()
                .username("user01")
                .email("user01@gmail.com")
                .password(
                    passwordEncoder.encode("123456")
                )
                .fullName("Nguyễn Hữu Trung")
                .images("/images/user.png")
                .role(userRole)
            );
        }
    };
}

```

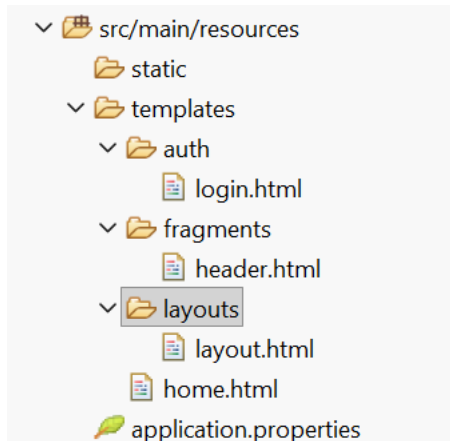
```

        .enabled(true)
        .build();

        userRepository.save(user);
    }
};
}

```

Bước 11: Tạo views



Home.html

```

<!DOCTYPE html>

<html lang="vi"
      xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorate="~{layouts/layout}">

<head>

    <title>Trang chủ</title>

</head>

<body>

<main layout:fragment="content">

    <h1>
        Trang chủ UTEShop
    </h1>

    <p>
        Xin chào
        <strong
            th:text="${#authentication.principal.fullName}">
            User
        </strong>
    </p>

```

```
</main>
```

```
</body>
```

```
</html>
```

Login.html

```
<!DOCTYPE html>
```

```
<html lang="vi"
      xmlns:th="http://www.thymeleaf.org">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <title>Đăng nhập</title>
```

```
    <meta name="viewport"
          content="width=device-width, initial-scale=1">
```

```
</head>
```

```
<body>
```

```
<h2>Đăng nhập</h2>
```

```
<div th:if="${param.error}">
```

```
    <p style="color:red">
```

```
        Username/email hoặc password không đúng
```

```
    </p>
```

```
</div>
```

```
<div th:if="${param.logout}">
```

```
    <p style="color:green">
```

```
        Bạn đã đăng xuất
```

```
    </p>
```

```
</div>
```

```
<form th:action="@{/Login}"
```

```
      method="post">
```

```
    <div>
```

```
        <label>
```

```
            Username hoặc Email
```

```
        </label>
```

```
        <input
```

```
            type="text"
```

```
            name="username"
```

```
            placeholder="Username hoặc email"
```

```
            required>
```

```
    </div>
```

```
<div>

    <label>
        Password
    </label>

    <input
        type="password"
        name="password"
        placeholder="Password"
        required>

</div>

<button type="submit">
    Đăng nhập
</button>

</form>

</body>
</html>
```

Layout.html

```
<!DOCTYPE html>

<html lang="vi"
    xmlns:th="http://www.thymeleaf.org"
    xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">

<head>

    <meta charset="UTF-8">

    <title layout:title-pattern="$CONTENT_TITLE - UTEShop">
        UTEShop
    </title>

</head>

<body>

<header th:replace="~{fragments/header :: header}">
</header>

<main layout:fragment="content">

</main>

<footer>

    Copyright © UTEShop
```

```
</footer>
```

```
</body>
```

```
</html>
```

Header.html

```
<!DOCTYPE html>
```

```
<html lang="vi"
      xmlns:th="http://www.thymeleaf.org">
```

```
<body>
```

```
<header th:fragment="header">
```

```
    <nav>
```

```
        <a th:href="@{/}">
```

```
            UTEShop
```

```
        </a>
```

```
        <div th:if="${#authorization.expression('isAuthenticated()')}">
```

```
            <!-- Avatar -->
```

```
            
```

```
            <!-- Full name -->
```

```
            <span
```

```
                th:text="${#authentication.principal.fullName}">
```

```
                Nguyễn Văn A
```

```
            </span>
```

```
            <!-- Username -->
```

```
            <span
```

```
                th:text="'(' + ${#authentication.principal.username} + ')'">
```

```
                (nguyenvana)
```

```
            </span>
```

```
            <!-- Email -->
```

```
            <span
```

```
                th:text="${#authentication.principal.email}">
```

```
                email@gmail.com
```

```

    </span>

    <!-- Role -->

    <span
        th:text="${#authentication.principal.role}">
        ROLE_USER
    </span>

    <!-- Logout -->

    <form
        th:action="@{/Logout}"
        method="post"
        style="display:inline">

        <button type="submit">
            Đăng xuất
        </button>

    </form>

</div>

<div th:unless="${#authorization.expression('isAuthenticated()')}">

    <a th:href="@{/Login}">
        Đăng nhập
    </a>

</div>

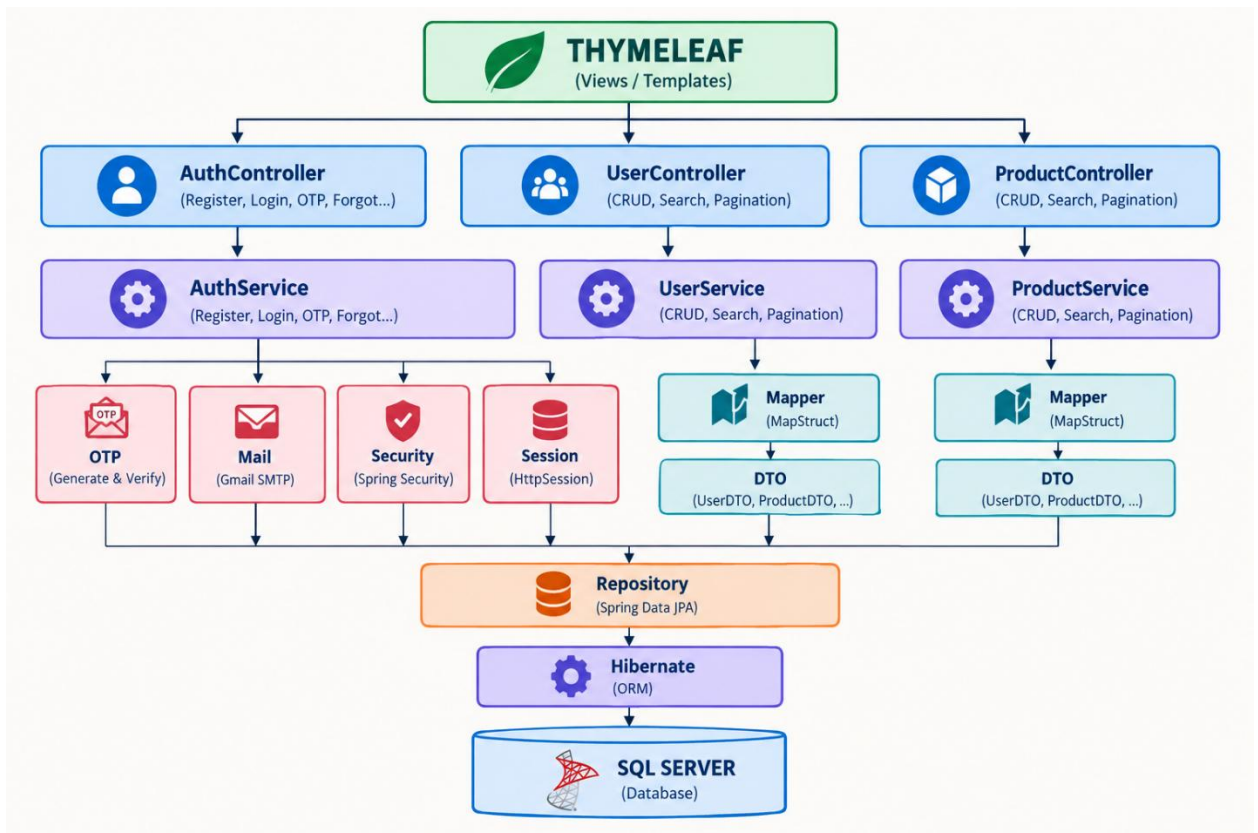
</nav>

</header>

</body>
</html>
=====

```

Ví dụ 3: Cho các bảng Users, Roles (user, admin), OtpToken, Products. Mối quan hệ 1 user - n product. Upload images vào cloud clouanry. Xây dựng code chi tiết cho chức năng Register xác nhận OTP qua mail, Login lưu session, forgotpassword gửi OTP qua mail, CRUD bảng user, product, tìm kiếm phân trang cho bảng user, cho bảng product, đếm số user, product của user bằng spring boot 4.1.1 + security mới nhất + mapper DTO to Entity và ngược lại bằng mapstruct, các view bằng thymeleaf + sql server.



Đọc file hướng dẫn Spring boot 4.docx nhé